Design and Implementation of a 64-bit Sequential RISC-V Processor

Verilog HDL Implementation

Vedant Zope (2024102003)
Kavya Pandey (2024102001)
Anushka Saini (2024102052)

Contents

1 Introduction **2**
1.1 Supported Instructions . . . 2
1.2 Design Overview . . . 2

2 Datapath Architecture **3**

3 Module Descriptions and Verilog Implementation **4**
3.1 Program Counter . . . 4
3.2 Instruction Memory . . . 4
3.3 Control Unit . . . 6
3.4 Register File . . . 7
3.5 Immediate Generator . . . 7
3.6 ALU Control . . . 8
3.7 64-bit ALU . . . 8
3.7.1 ALU Operations . . . 9
3.8 Data Memory . . . 9
3.9 Test Program 1 — Sum of First N Natural Numbers . . . 9
3.10 Register File — Sum of N . . . 10
3.11 Test Program 2 — Fibonacci Sequence . . . 10
3.12 Register File — Fibonacci . . . 10

4 Timing Analysis **11**
4.1 Critical Path for Each Instruction . . . 11
4.2 Choosing the Clock Period . . . 11
4.3 ALU Delay Breakdown . . . 11

5 Read/Write Timing and Clocked Writes **12**
5.1 When Do Reads and Writes Happen? . . . 12
5.2 Why Writes Are Clocked (Not Asynchronous) . . . 12

6 Design Decisions and Trade-offs **12**

7 File Structure **13**

8 Conclusion **13**

1 Introduction

This report covers the complete design, Verilog implementation, and timing analysis of a 64-bit sequential processor based on the RISC-V RV64I base integer instruction set. In a single-cycle architecture, every instruction completes within exactly one clock cycle(except the write operation), and the program counter (PC) advances at every rising clock edge.

The architecture follows the standard sequential datapath model and includes the PC, Instruction Memory, Register File, Control Unit, Immediate Generator, ALU Control, ALU, Data Memory, multiplexers, and adder blocks. The design is modular, with each stage implemented as a separate Verilog module and integrated to form the complete processor.

1.1 Supported Instructions

- **R-type** (register–register): `add`, `sub`, `and`, `or`,
- **I-type** (immediate): `addi`
- **Load**: `ld` (64-bit doubleword load)
- **Store**: `sd` (64-bit doubleword store)
- **Branch**: `beq` (branch if equal)

1.2 Design Overview

The processor is composed of the following functional units connected via a unified datapath:

1. Program Counter (PC)
2. Instruction Memory (4096 bytes)
3. Control Unit
4. Register File (32×64-bit)
5. Immediate Generator
6. ALU Control
7. 64-bit ALU (10 sub-modules)
8. Data Memory (1024 bytes)

All blocks operate *combinationally* within a single clock period; only the PC register, register file writes, and data memory writes are clocked (synchronous).

2 Datapath Architecture

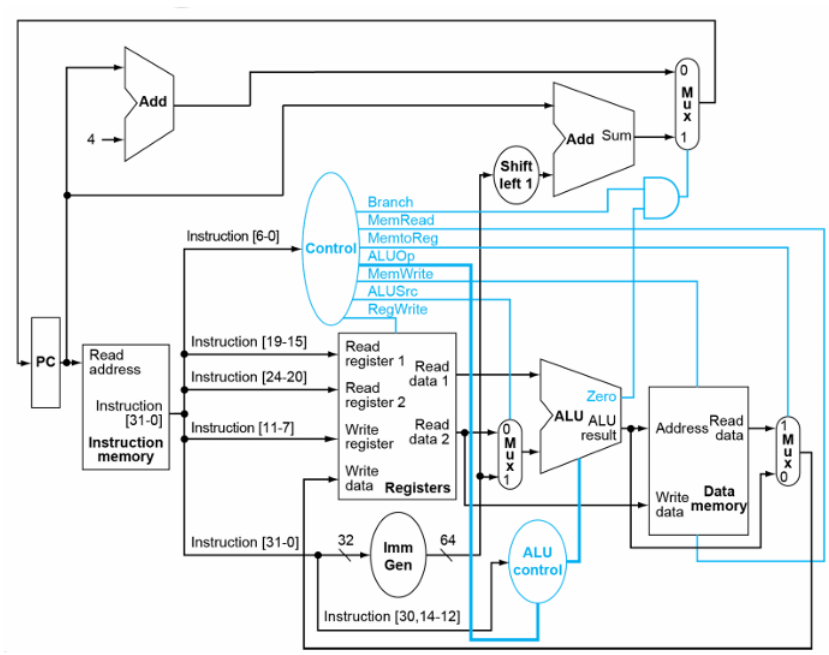

Figure 1: Enter Caption

The datapath refers to the sequential flow through which an instruction is executed. The following blocks each perform a specific task and, upon integration, together form the complete processor:

1. **Instruction Fetch (IF)**

   The PC addresses the instruction memory, which outputs a 32-bit instruction assembled from four big-endian bytes.

2. **Instruction Decode (ID)**

   The 7-bit opcode is sent to the Control Unit, which generates all required control signals. Simultaneously, the `rs1` and `rs2` fields address the Register File, and the Immediate Generator extracts and sign-extends the embedded immediate value.

3. **Operation (OP)**

   The ALU Control unit derives a 4-bit control signal from `ALUOp`, `funct3`, and `funct7[5]`. A multiplexer selects the second ALU operand (register data or immediate). The 64-bit ALU then performs the required arithmetic or logical operation.

4. **Memory Access (MEM)**

   For `ld`, the ALU result is used as the memory address and 8 bytes are read. For `sd`, the data from `rs2` is written to the ALU-computed address. All other instructions bypass this stage.

5. **Write-Back (WB)**

A multiplexer selects between the ALU result and the memory read data. The selected value is written to register `rd` when `RegWrite` is asserted.

6. **PC Update**

$$\texttt{PC_next} = \begin{cases} \texttt{PC} + \texttt{imm}, & \text{if } (\texttt{Branch} \wedge \texttt{zero_flag}) \\ \texttt{PC} + 4, & \text{otherwise} \end{cases}$$

The immediate value is added only when the `beq` instruction evaluates to true, causing the Program Counter to branch to the target address specified in the instruction.

3 Module Descriptions and Verilog Implementation

3.1 Program Counter

A 64-bit register that resets to zero and updates to `pc_in` on every rising clock edge. The next-PC logic uses two adders (PC+4 and PC+imm) and a mux controlled by `Branch` $\wedge$ `zero_flag`.

The program counter *points* at the instruction which is set in the instruction memory, which is changed at the rising clock edge. That particular instruction is then decoded and carried out by the processor.

3.2 Instruction Memory

A byte-addressable memory array of 4096 bytes is used to store program instructions. The memory contents are initialized at the start of simulation by reading hexadecimal byte values from the file `instructions.txt`. Since the memory is byte-addressed and follows a big-endian format, four consecutive bytes are concatenated in big-endian order to construct a single 32-bit instruction.

Each 32-bit instruction is structured according to the RISC-V instruction encoding format. Specific bit fields within the instruction—such as the opcode, register specifiers (`rs1`, `rs2`, `rd`), `funct3`, `funct7`, and immediate fields—are extracted and routed to the appropriate functional blocks. The instruction decoding depends on the type of instruction.

The opcode field is routed to the Control Unit to generate the appropriate control signals. The register indices (`rs1`, `rs2`, and `rd`) are sent to the Register File for operand access and destination selection. The `funct3` and `funct7` fields are provided to the ALU Control unit, which determines the specific arithmetic or logical operation to be performed.

- **R-type instruction**
  R-type instructions perform arithmetic or logical operations using two source registers (`rs1`, `rs2`) and produce a result in the destination register (`rd`). The Register File reads the values of `rs1` and `rs2` and provides them to the ALU. The ALU operation is determined by the control signals derived from the instruction fields. The computed result is written back to `rd`.

funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**
[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]

Table 1: R-Type Instruction Format (RV64I)

Instruction	opcode [6:0]	funct3 [14:12]	func7_30[30]	ALUControl	Operation
add	0110011	000	0	0000	ADD
sub	0110011	000	1	1000	SUB
and	0110011	111	X	0111	AND
or	0110011	110	X	0110	OR
xor	0110011	100	X	0100	XOR
sll	0110011	001	X	0001	SLL
srl	0110011	101	0	0101	SRL
sra	0110011	101	1	1101	SRA
slt	0110011	010	X	0010	SLT
sltu	0110011	011	X	0011	SLTU

Table 2: Instruction Mapping for R-Type Instructions According to ALU Operation

- **I-type instruction**

  I-type instructions perform arithmetic or logical operations using one source register (`rs1`) and an immediate value (`imm`). The Register File provides the value of `rs1`, while the Immediate Generator supplies the immediate operand. A multiplexer selects between the immediate and register input for the ALU. The ALU operation is determined by the control signals, and the result is written back to `rd`.

immediate	rs1	funct3	rd	opcode
[31:20]	[19:15]	[14:12]	[11:7]	[6:0]

Table 3: I-Type Instruction Format (RV64I)

Instruction	opcode [6:0]	funct3 [14:12]	bit30 [30]	Immediate Source	ALUControl	Operation
addi	0010011	000	X	instr[31:20] (signed)	0000	ADD
slli	0010011	001	0	instr[24:20] (shamt)	0001	SLL
slti	0010011	010	X	instr[31:20] (signed)	0010	SLT
sltiu	0010011	011	X	instr[31:20] (signed)	0011	SLTU
xori	0010011	100	X	instr[31:20] (signed)	0100	XOR
srli	0010011	101	0	instr[24:20] (shamt)	0101	SRL
srai	0010011	101	1	instr[24:20] (shamt)	1101	SRA
ori	0010011	110	X	instr[31:20] (signed)	0110	OR
andi	0010011	111	X	instr[31:20] (signed)	0111	AND

Table 4: Instruction Mapping for I-Type Arithmetic Instructions

- **Load (doubleword)**
  The load instruction reads a value from data memory and writes it into the destination register `rd`. The effective memory address is computed as:

$$\text{Address} = \texttt{rs1} + \texttt{offset}$$

  The value stored at this computed address is then read from memory and written into `rd`. In this RV64I implementation, the load operation transfers a doubleword (64 bits) from memory. Load is a I-type instruction, but behaves differently than its counterparts.

- **Store (doubleword)**

immediate	rs1	funct3	rd	opcode
[31:20]	[19:15]	[14:12]	[11:7]	[6:0]

Table 5: Load (ld) Instruction Format (RV64I)

The store instruction writes a value from a source register into data memory. The effective memory address is computed as:

$$\text{Address} = \texttt{rs1} + \texttt{offset}$$

The value stored in `rs2` is written to the computed memory address. In this RV64I implementation, the store operation transfers a doubleword (64 bits) from the register file to memory. Store is a S-type instruction.

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]

Table 6: Store (sd) Instruction Format (V64I)

- **Branch instruction**
  The `beq` instruction performs a conditional branch based on the comparison of two source registers, `rs1` and `rs2`. The values stored in these registers are read from the Register File and compared using the ALU.
  If the two values are equal, the Zero flag is asserted, and the Program Counter (PC) is updated to the branch target address.
  If the values are not equal, the branch is not taken, and the PC increments normally by 4.

$$\text{PC}_{\text{next}} = \begin{cases} \text{PC} + \text{offset}, & \text{if } \texttt{rs1} = \texttt{rs2} \\ \text{PC} + 4, & \text{otherwise} \end{cases}$$

  The immediate offset used for branching is sign-extended and added to the current PC to compute the target address.

imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]

Table 7: B-Type Instruction Format (`beq`, RV64I)

3.3 Control Unit

A purely combinational decoder that maps the 7-bit opcode to seven control signals. All signals default to 0, preventing latch inference.

- **Branch**: Asserted for branch instructions, when the branching condition is deemed true. When high and the Zero flag is set, the PC is updated to branch target address.
- **MemRead**: Enables reading data from memory. Used in load instructions to fetch data from the computed address.
- **MemtoReg**: It is the select line for the MUX which decides whether the data read from the memory or the ALU output is supposed to be written in the register(rd).
- **MemWrite**: Enables writing data to memory. Used in store instructions to write register data to the computed address.

- **ALUSrc**: Selects the second ALU operand. When high, the ALU uses the immediate value instead of `rs2`.
- **RegWrite**: Enables writing to the Register File. When high, the result of the ALU or memory is written into `rd`.
- **ALUOp**: A 2-bit signal that determines the ALU operation type. It is used by the ALU Control unit along with instruction fields to generate the final ALU control signal.

Control Signal	R-type	I-type (addi)	Load (ld)	Store (sd)
Branch	0	0	0	0
MemRead	0	0	1	0
MemtoReg	0	0	1	0
MemWrite	0	0	0	1
ALUSrc	0	1	1	1
RegWrite	1	1	1	0
ALUOp	10	00	00	00

Table 8: Control Signals based on the type of the instruction (RV641)

3.4 Register File

Contains 32 general-purpose 64-bit registers with read and write capabilites.
The registers are read asynchronously, independent of what instruction is being executed. The registers are written synchronously, i.e. only on the positive clock edge and the control signal RegWrite is asserted.

Writes to register x0 are blocked.

3.5 Immediate Generator

It is a combinational logic block responsible for extracting and formatting the immediate field from the 32-bit instruction. Since different instruction formats (I-type, S-type, and B-type) encode immediate values in different bit positions, the block rearranges and concatenates the required instruction bits accordingly.

The generated immediate is sign-extended to 64 bits to match the RV64I datapath width. This ensures correct handling of negative offsets and signed arithmetic operations.

- The generated 64-bit sign-extended immediate is routed to the second input of the ALU through a MUX. When `ALUSrc` is asserted, the MUX selects the immediate value, allowing the ALU to perform immediate-based arithmetic operations and effective address calculations.
- For branch instructions, the immediate is also passed to a shift-left-by-1 block. This shift aligns the branch offset properly for PC-relative addressing. The shifted immediate is added to the current PC to compute the branch target address.
  A final MUX selects between `PC + 4` (sequential execution) and the computed branch target address. The selected value becomes `PC_next`.

3.6 ALU Control

Translates `ALUOp`, `funct3`, and `funct7[5]` into a 4-bit `ALUControl` signal which defines which operation to be performed by the ALU.

ALUOp	funct3	funct7_30	ALUControl	Operation
00	XXX	X	0000	ADD (Load/Store)
01	XXX	X	1000	SUB (Branch comparison)
10	000	0	0000	ADD
10	000	1	1000	SUB
10	111	X	0111	AND
10	110	X	0110	OR

Table 9: ALU Control Signal Generation Logic

In this processor desgin, only ADD, SUB, AND, OR is necessary to be performed by the ALU and therefore only those decoding is implemented. But the ALU is capable of performing the following operations based on ALU-Control it recieves after the coding.

ALUControl	Operation
0000	ADD
0001	SLL
0010	SLT
0011	SLTU
0100	XOR
0101	SRL
0110	OR
0111	AND
1000	SUB
1101	SRA

Table 10: ALU Operation Selection Based on 4-bit ALUControl

3.7 64-bit ALU

The ALU instantiates 10 dedicated sub-modules that all compute in parallel. A 10-to-1 mux selects the correct result based on `ALUControl`.

Arithmetic Logic Unit (ALU)

The ALU takes two 64-bit inputs: the value from `rs1` and a second operand selected by a MUX between `rs2` and the `imm` . The selected operands are processed according to the 4-bit `ALUControl` signal.

The ALU output is routed either to the Data Memory (for load and store address computation) or directly to the Register File for write-back, depending on the instruction type.

In addition to the result, the ALU generates status flags:

- **Carry flag**: Generated only during addition and subtraction operations.
- **Overflow flag**: Indicates signed overflow in addition and subtraction operations.

- **Zero flag**: Asserted when the ALU result equals zero.

These flags are used for the monitoring of arithmetic conditions and for control decisions such as branch evaluation.

3.7.1 ALU Operations

- **ADD** (`add_mod`): 64-bit addition implemented using ripple-carry logic.
- **SUB** (`sub_mod`): 64-bit subtraction performed using two's complement addition.
- **AND / OR / XOR** (`and_mod / or_mod / xor_mod`): Bitwise operations implemented using `generate` loops.
- **SLL** (`sll_mod`): Logical left shift, fills vacated least significant bits with 0.
- **SRL** (`srl_mod`): Logical right shift, fills vacated most significant bits with 0.
- **SRA** (`sra_mod`): Arithmetic right shift, fills vacated most significant bits with `a[63]` (sign bit).
- **SLT** (`slt_mod`): Signed set-less-than. Subtracts operands and checks sign bits.
- **SLTU** (`sltu_mod`): Unsigned comparison using carry-out from subtraction.

3.8 Data Memory

It is implemented as a 1024-byte, byte-addressable memory with a 10-bit address bus and a 64-bit data bus, which is capable of reading and writing memory addresses.

When `MemWrite` is asserted, the 64-bit value on `write_data` is written to the specified address. This action is synchronous with the positive clock edge.
When `MemRead` is asserted, the 64-bit value stored at the given address is provided on `read_data`. This action is asynchronous.

3.9 Test Program 1 — Sum of First N Natural Numbers

Table 11: Test Program 1 — Sum of $N = 10$ (42 Cycles)

Cycle	PC	Instruction	Type
1	0x00	`addi x1, x0, 10`	I-type
2	0x04	`addi x2, x0, 0`	I-type
3	0x08	`addi x3, x0, 1`	I-type
4	0x0C	`add x2, x2, x1`	R-type (ALU ADD)
5	0x10	`sub x1, x1, x3`	R-type (ALU SUB)
6	0x14	`beq x1, x0, +8`	B-type (not taken)
7	0x18	`beq x0, x0, -12`	B-type (taken → 0x0C)
— loop repeats 9 more times (cycles 4–7) —			
41	0x14	`beq x1, x0, +8`	B-type (taken → 0x1C)
42	0x1C	`beq x0, x0, 0`	Infinite loop (halt)

3.10 Register File — Sum of N

Table 12: Register State After Sum-of-N Simulation (42 Cycles)

Reg	Hex Value	Decimal
x0	0000000000000000	0
x1	0000000000000000	0
x2	0000000000000037	55
x3	0000000000000001	1
x4–x31	0000000000000000	0

Verification: $\dfrac{N(N+1)}{2} = \dfrac{10 \times 11}{2} = 55$

3.11 Test Program 2 — Fibonacci Sequence

Table 13: Test Program 2 — Fibonacci (51 Cycles)

Cycle	PC	Instruction	Type
1	0x00	`addi x1, x0, 0`	I-type
2	0x04	`addi x2, x0, 1`	I-type
3	0x08	`addi x4, x0, 8`	I-type
4	0x0C	`addi x5, x0, 1`	I-type
5	0x10	`add x3, x1, x2`	R-type (ALU ADD)
6	0x14	`add x1, x2, x0`	R-type (ALU ADD)
7	0x18	`add x2, x3, x0`	R-type (ALU ADD)
8	0x1C	`sub x4, x4, x5`	R-type (ALU SUB)
9	0x20	`beq x4, x0, +8`	B-type (not taken)
10	0x24	`beq x0, x0, -20`	B-type (taken → 0x10)
— loop repeats 7 more times (cycles 5–10) —			
50	0x20	`beq x4, x0, +8`	B-type (taken → 0x28)
51	0x28	`beq x0, x0, 0`	Infinite loop (halt)

3.12 Register File — Fibonacci

Table 14: Register State After Fibonacci Simulation (51 Cycles)

Reg	Hex Value	Decimal
x0	0000000000000000	0
x1	0000000000000015	21
x2	0000000000000022	34
x3	0000000000000022	34
x4	0000000000000000	0
x5	0000000000000001	1
x6–x31	0000000000000000	0

Verification — Expected Fibonacci Sequence:

$$\underbrace{0}_{F_0}, \underbrace{1}_{F_1}, \underbrace{1}_{F_2}, \underbrace{2}_{F_3}, \underbrace{3}_{F_4}, \underbrace{5}_{F_5}, \underbrace{8}_{F_6}, \underbrace{13}_{F_7}, \underbrace{21}_{F_8}, \underbrace{34}_{F_9}$$

Final register values `x1` $= 21 = F(8)$ and `x2` $= 34 = F(9)$ match the expected sequence.

All values match the expected results.

4 Timing Analysis

4.1 Critical Path for Each Instruction

Every instruction goes through a different set of components. The delay along the longest path decides how fast we can clock the processor.

Table 15: Critical Path Comparison

Type	**Critical Path**	**Stages**
R-type	$t_{\text{CQ}} + t_{\text{IM}} + t_{\text{RF,r}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{mux}} + t_{\text{RF,su}}$	IF, ID, EX, WB
I-type	$t_{\text{CQ}} + t_{\text{IM}} + \max(t_{\text{RF,r}}, t_{\text{IG}}) + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{mux}} + t_{\text{RF,su}}$	IF, ID, EX, WB
`ld`	$t_{\text{CQ}} + t_{\text{IM}} + t_{\text{RF,r}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{DM,r}} + t_{\text{mux}} + t_{\text{RF,su}}$	IF–WB
`sd`	$t_{\text{CQ}} + t_{\text{IM}} + t_{\text{RF,r}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{DM,su}}$	IF–MEM
`beq`	$t_{\text{CQ}} + t_{\text{IM}} + t_{\text{RF,r}} + t_{\text{ALU}} + t_{\text{mux}}$	IF, ID, EX

t_{CQ}: PC clock-to-Q t_{IM}: instruction memory $t_{\text{RF,r}}$: register read t_{IG}: imm gen t_{mux}: MUX t_{ALU}: ALU $t_{\text{DM,r}}$: data mem read $t_{\text{RF,su}}/t_{\text{DM,su}}$: setup time.

4.2 Choosing the Clock Period

Since this is a single-cycle design, every instruction has to finish in one clock period. The clock is only as fast as the slowest instruction. Looking at the table, `ld` clearly takes the longest—it is the only one that goes through all five stages. So the minimum clock period is:

$$\boxed{T_{\text{clk}} \;\geq\; t_{\text{CQ}} + t_{\text{IM}} + t_{\text{RF,r}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{DM,r}} + t_{\text{mux}} + t_{\text{RF,su}}}$$

Every other instruction finishes sooner but still has to wait for the full T_{clk}.

4.3 ALU Delay Breakdown

Table 16: ALU Sub-Module Delays

Module	**Architecture**	**Delay**
AND / OR / XOR	1-gate deep	$O(1)$
SLL / SRL / SRA	6-stage barrel shifter	$O(\log n)$
ADD / SUB	64-bit ripple carry	$O(n)$
SLT / SLTU	Subtract + compare	$O(n)$

The ripple-carry adder is the ALU bottleneck: the carry propagates through all 64 bits serially. Since all 10 sub-modules compute in parallel, the overall t_{ALU} is bounded by the adder/subtractor.

5 Read/Write Timing and Clocked Writes

5.1 When Do Reads and Writes Happen?

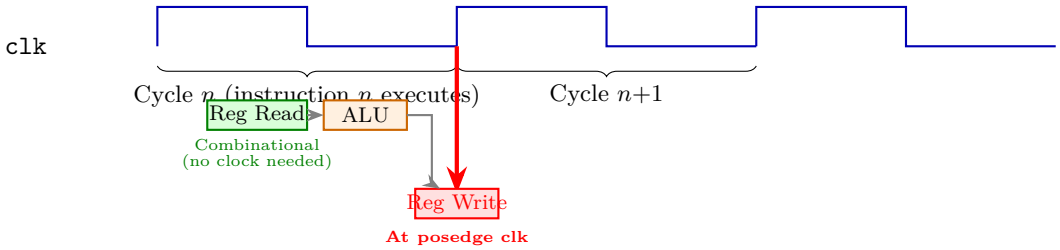

Figure 2: Read/write timing for R-type and I-type instructions. Register reads happen combinationally (early in the cycle); register writes happen only at the rising clock edge.

- **Register reads are combinational**—the register file outputs data as soon as the address lines (`rs1`, `rs2`) are available from the instruction. No clock edge needed.
- **Register writes happen at the positive clock edge**—the result computed by the ALU (or loaded from memory) gets written into `rd` only when the clock transitions from 0 to 1. Until that edge, the register still holds its old value.
- If an instruction reads and writes the *same* register in one cycle, the read returns the old value and the new value is stored at the end. This works because the write waits for the edge.

5.2 Why Writes Are Clocked (Not Asynchronous)

Both the register file and data memory use **synchronous (clocked) writes**—data is latched only at `posedge clk`. The writes are *synchronous*.

During the cycle, signals are still propagating through the combinational logic and may temporarily hold wrong values (glitches). If the register file wrote data the instant it appeared at the input, those glitches would get stored. By making the write wait for the clock edge, we make sure the data has fully settled to its correct value before anything is actually stored.
The same applies to data memory—`sd` computes the address through the ALU, but the actual memory write only happens at the clock edge, not before.

6 Design Decisions and Trade-offs

1. **Single-cycle vs. pipelined**: Simplifies control (no hazard detection, no forwarding) but limits $f_{\max}$ based on the slowest instruction (`ld`).
2. **Structural ALU**: Each operation has a dedicated module. All compute simultaneously, but this provides good modularity and independent testability.
3. **Big-endian memory**: Both memories use big-endian byte ordering.
4. **x0 protection**: Write to `x0` is blocked in the register file, as required by RISC-V.

7 File Structure

Table 17: Project Files

File	Purpose
`proc.v`	Top-level processor
`control_unit.v`	Opcode decoder
`registers.v`	32×64-bit register file
`instruction_memory.v`	4096-byte instruction ROM
`data_memory.v`	1024-byte data RAM
`immgen.v`	Immediate sign-extension
`alucon.v`	ALU control decoder
`alu.v`	ALU top-level (10-op mux)
`add_mod.v` ... `sra_mod.v`	10 ALU sub-modules
`seq_tb.v`	Testbench
`instructions.txt`	Hex-encoded test program
`register_file.txt`	Simulation output

8 Conclusion

This report covers the complete design, Verilog implementation, and timing analysis of a 64-bit single-cycle RISC-V processor (RV64I subset). The modular design—with a dedicated control unit, 64-bit ALU composed of 10 structural sub-modules, register file with x0 protection, and dual memory subsystem—provides a working demonstration of core processor concepts.

Timing analysis shows that the `ld` instruction determines the minimum clock period since it traverses all five datapath stages. The register file and data memory use synchronous writes to prevent glitches from corrupting stored data; reads are combinational and happen during the cycle, while writes take effect only at the positive clock edge. The ripple-carry adder represents the primary ALU bottleneck at $O(n)$ delay, while the barrel shifters provide $O(\log n)$ shifting.

The test program exercises all supported instruction types, and the simulation confirms correct execution across 15 clock cycles.